Smoke & Regression Testing

Build Validation • Release Safety Nets • Automation Strategy

Brian Scott McCarthy | Senior QA Lead | QA Best Practice Series — BP-05

1. Smoke Testing — Build Validation

> **What is a Smoke Test?** A fast, shallow test suite that validates the most critical functions of a build are working before the team invests time in deeper testing. If the smoke test fails, the build is rejected immediately — no further testing proceeds.

Attribute	Smoke Test	Full Regression
Purpose	Is the build stable enough to test?	Does the release break anything?
Scope	Critical-path scenarios only (P1)	All P1 + P2 + P3 test cases
Depth	Shallow — verify features work, not edge cases	Deep — including boundary, negative, edge cases
Duration	10–20 minutes (automated)	1–4 hours (automated full suite)
When to run	Every build deployed to QA env	Every sprint end; every release candidate
Trigger	Automated: on every deployment	Automated: nightly + on PR to main; manual: pre-release
Failure action	Reject build; notify dev team; halt QA	Triage failures; log bugs; assess Go/No-Go

2. What Goes in a Smoke Test Suite

Smoke tests should cover the absolute minimum that must work for the application to be usable. Aim for 15–25 test cases maximum.

Category	Example Smoke Test Cases
App loads	Homepage loads without JS errors; HTTP 200; above-the-fold content visible within 3s
Authentication	Register new account; login with valid credentials; logout clears session
Core navigation	Main nav links resolve to correct pages; back button works; 404 page for bad URL
Primary user action	Complete the #1 user journey (e.g. add item to cart + checkout; or submit a form; or book a session)
API health	Critical GET endpoints return 200 with correct schema (automated Postman/REST Assured)
Data persistence	Create a record; refresh; record still exists (DB round-trip)
Authentication gates	Unauthenticated user cannot access /dashboard, /account, /admin
Email delivery	Trigger a transactional email; verify received within 2 minutes (Mailtrap)

3. Regression Testing Strategy

Regression Type	Scope	When	Duration Target
Smoke Regression	P1 critical path only	Every deployment to QA	< 20 min automated
Partial Regression	Changed modules + directly dependent modules	Every PR merge to main branch	< 45 min automated

Regression Type	Scope	When	Duration Target
Sprint Regression	All sprint test cases + P1-P2 from previous sprints	End of each sprint	1–2 hrs automated + manual exploratory
Full Regression	Entire test suite — all modules, all priorities	Pre-release / release candidate	2–4 hrs automated; 1 day manual coverage
Hotfix Regression	Impacted area only + smoke	After any production hotfix	15–30 min — fast turnaround

4. Selecting Tests for Regression — Decision Rules

Not every test case belongs in regression. Use these rules to build and maintain a lean, high-value suite:

Include in Regression?	Criteria
Always include	P1 critical-path scenarios; any test that caught a production defect in the last 6 months
Include if stable	P2 scenarios for core features; tests with < 5% flake rate over last 10 runs
Review quarterly	P3 scenarios; tests for deprecated or infrequently used features
Remove from suite	Tests with > 10% flake rate that have not been fixed; tests for features no longer in the product
Never include	Exploratory testing outcomes; one-time validation; tests requiring manual human judgment

5. Automation for Smoke & Regression

Framework	Role in Smoke/Regression	Command
Playwright	Primary regression suite; cross-browser; E2E flows	npx playwright test --project=chromium (smoke) npx playwright test (full regression)
Playwright — tagged	Smoke subset tagged @smoke	npx playwright test --grep @smoke
Selenium (Java)	Legacy regression; multi-environment parallel	mvn test -Dgroups=smoke (TestNG groups)
Postman / Newman	API smoke and regression	newman run Smoke_Suite.json -e qa.json
Cypress	Component-level regression; fast feedback	npx cypress run --spec 'cypress/e2e/smoke/**'

Tagging strategy: Tag every test case with @smoke, @regression, @p1, @p2, and @module-name. This allows targeted runs (e.g. only smoke tests, only checkout module) without maintaining separate files.

6. Go / No-Go Decision Framework

Condition	Recommendation
Smoke suite: 100% pass; 0 P1 bugs open	GO — deploy to next environment
Smoke suite: 1 P2 failure; workaround documented	CONDITIONAL GO — Product Owner must accept risk in writing

Condition	Recommendation
Smoke suite: any P1 failure	NO-GO — reject build; notify Dev Lead; do not proceed with further testing
Full regression: ≥ 95% pass; all P1+P2 resolved	GO for production
Full regression: < 95% pass OR any open P1 bug	NO-GO — defects must be resolved or formally accepted before release
Known issue with approved workaround	CONDITIONAL GO — document in release notes; notify Customer Support

7. Regression Maintenance — Keeping the Suite Healthy

- Review and remove obsolete tests every sprint — a bloated suite is slower and harder to trust.
- Quarantine flaky tests within 24 hours of first flaky detection — do not let them mask real failures.
- Update test cases immediately when UI or API contracts change — never leave broken tests in the suite.
- Review automation coverage metrics in every sprint review — coverage should grow sprint-over-sprint.
- Pair every new feature with at least one automation-candidate test case before sprint closure.
- Track flaky test rate as a KPI — target < 2% of suite flaking per run. Investigate root cause above threshold.